

Detecting Textual Phishing Contents Using LLM-based System

GitHub Repository of This Project: https://github.com/joting0518/cyber_LLM

楊大明
National Chengchi
University
111701043

林佩璇
National Chengchi
University
111302055

陳若庭
National Chengchi
University
111306011

李宜恩
National Chengchi
University
111208001

Absrtact

As the development of Large Language Models, hackers obtain the capability to generate high quality phishing BEC emails with low cost. This causes a serious challenge to cyber security. To deal with this issue, this study built a LLM-based detection system, which can detect AI-generated phishing contents in emails.

In this paper, we built a LLM-based WebUI tool that can detect phishing contents. For the dataset to train the model, we incorporated real-world legitimate emails, conventional phishing emails, and AI-generated phishing emails. The diverse dataset enables the model to detect not only human-crafted phishing attacks but also AI-generated contents, thereby improving its effectiveness against AI-assisted phishing threats. The experimental result using Qwen2.5-1.5B-Instruct demonstrates outstanding performance, with a precision of 0.9984 and a recall of 0.9936. Especially for the precision, it has a great increase from 0.5416 to 0.9984, which indicates that the fine-tuning process has decreased the false positive effectively.

These findings highlight the potential of fine-tuning techniques on detecting AI-generated phishing contents. From the change of precision before and after fine-tuning, it also indicates that the system can effectively decrease the risk of alarm fatigue.

1 Introduction

Phishing attacks have long been a significant issue in the field of cybersecurity. As attack techniques continue to evolve, defenders have also continuously updated their defense strategies. Early approaches primarily relied on machine learning for pattern recognition[1], while the development of LLMs in recent years has led to the emergence of defense strategies that leverage LLMs for phishing detection.

A previous study demonstrated the development of LLM-based systems for phishing email detection. [2] Through prompt engineering techniques, the study achieved F1-scores of 85.4% with Mistral-7B and 80.1% with LLaMA3-8B. Furthermore, LLMs have also been applied to phishing website detection. A prior system utilizing ChatGPT for phishing website detection achieved a precision of 98.7% and a recall of 99.6% when using GPT-4V. [3]

Although defenders continue to improve their detection capabilities, the advancement of LLMs has also provided attackers with opportunities to reduce attack costs, enhance attack effectiveness, and launch larger-scale and more harmful attacks. [4] Previous experiments have shown that among Business Email Compromise (BEC) phishing emails generated by LLMs, 86.4% successfully evaded Gmail's detection mechanisms. [5] A trends report by Hoxhunt also points out that they have uncovered a 14 times surge in AI-generated phishing that could bypass email filters. [6] This situation highlights the need for continued research into effective defense strategies to AI-generated phishing emails.

Based on the above, we developed a system that could identify both conventional and AI-generated phishing attacks. We adopted Qwen2.5-1.5B-Instruct, fine-tuned it by using the LoRA method, and finally built a user-friendly WebUI that enables users to conveniently analyze suspicious emails.

In summary, we make the following contributions:

- We propose a system for detecting conventional and AI-generated phishing emails.
- We conducted an experiment using the LoRA fine-tuning method, and achieved a high precision of 0.9984 and a recall of 0.9936. This shows that fine-tuning techniques can enhance the capabilities of identifying phishing or not.

Table 1: Description of Column in the Combined Dataset

Column	Type	Description
subject	String	The email subject line after preprocessing and normalization.
body	String	The email body content after HTML removal, whitespace normalization, and text cleaning.
label	Integer	Binary classification label. 0 represents a legitimate email, while 1 represents a phishing email.
dataset	Integer	Source dataset identifier used for analysis and error tracking.

2 Dataset description

We collected three datasets: The Enron Email Dataset¹, Nazario Phishing Corpus², and Adversarial BEC Email Dataset³.

The **Enron Email Dataset** is a real-world dataset which contains approximately 500,000 emails generated by the employees of the Enron Corporation. In our system, it stands for legitimate emails.

As for phishing emails, we chose Nazario Phishing Corpus and Adversarial BEC Email Dataset. **Nazario dataset** contains phishing emails collected from 2015 to 2025. It is a representative dataset of phishing emails, since usually used in past papers that discuss features of phishing emails, social engineering techniques, and detection automation.

However, discussing classic phishing emails is not enough. Since there will be more and more phishing attacks generated by LLMs, we believe that our system needs to obtain the ability for detecting new types of phishing. Therefore, we obtained **Adversarial BEC Email** in our dataset. The content is generated by Gemini Flash 2.5, and it contains emails processed by adversarial modification. BEC attack usually attacks by disguising administrators, distributors, and partners without providing URLs or attachments. For these issues, this kind of attack activities are seen as one of the threatening email attacks.

2.1. Data Cleaning

In this step, we processed the datasets separately, then combined them into one dataset. The process is stored as github repository⁴. Table 1 at the next page shows the schema of the processed dataset.

There were 517,401 rows of data in the Enron Email Dataset. We removed emails whose subject or body is empty, and also removed emails that are too short and duplicated. After the process, we obtained 228,221 emails. We set their label as 0 since this dataset is for legitimate, and their dataset column are set as 1, to make sure they are from Enron Email dataset.

For Nazario and Adversarial BEC Email, we use the same process to clean the data. After cleaning, we obtained 950 emails in the former and 5,338 emails in the latter. As for label column and dataset column, we set their label as 1 since they are phishings, and 2 for the former, 3 and 4 for the latter.

2.2. Dataset Balancing and Split

After data cleaning, we find that the amount of legitimate emails are far more than phishing emails. To avoid the imbalance, all phishing emails were retained and an equal number of legitimate emails were randomly sampled with the random seed being 42. Finally, our dataset contains 6,288 phishing and legitimate emails, and total size is 12,576. The dataset split is performed as Table 2 shows, with the random seed being 42.

¹ The Enron Email Dataset: <https://www.kaggle.com/datasets/wcukierski/enron-email-dataset/data>

² Nazario Phishing Corpus: <https://monkey.org/~jose/phishing>

³ Adversarial BEC Email Dataset: <https://www.kaggle.com/datasets/yoedjei/adversarial-bec-email-dataset/data>

⁴ Github repository of data cleaning: https://github.com/joting0518/cyber_LLM

Table 2: Dataset Split

Dataset	Split ratio	Size	Label=1	Label=0
train.csv	80%	10,060	5,030	5,030
valid.csv	10%	1,258	629	629
test.csv	10%	1,258	629	629

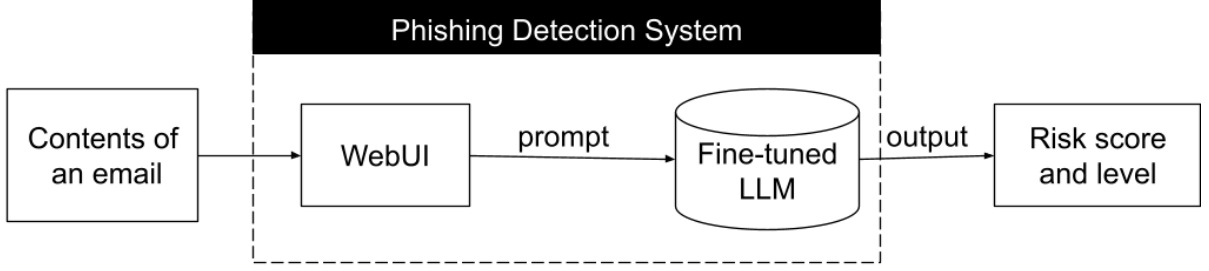


Figure 1: Overview of System Architecture

3 Experiments

3.1. Overview

The proposed system adopts a browser-based architecture to ensure cross-platform compatibility between different OS. As illustrated in figure 1, the system consists of a web user interface and a fine-tuned LLM.

This architecture provides a lightweight solution while leveraging the analysis capabilities of LLM for phishing detection.

3.2. Model

In this project, we used **ollama qwen2.5:1.5b** as a baseline model, and the fine-tuned model we called Qwen2.5-1.5B-Instruct after LoRA BF16 fine-tuning.

For the model, we expect it to classify the emails that are given by the user whether they are phishing or not. Therefore, the system prompt is set as Figure 2.

```

messages = [
  {
    "role": "system",
    "content": (
      "You are a cybersecurity email classifier. "
      "Decide whether the email is phishing or legitimate. "
      "Do not copy the email. Do not explain. "
      "Output only one label: phishing or legitimate."
    ),
  },
  {
    "role": "user",
    "content": "Subject: ...\nBody: ...",
  },
]

```

Figure 2: System Prompt

3.3. Fine-Tuning Approach

With Qwen/Qwen2.5-1.5B-Instruct as the base, this training felt more like careful refinement than aggressive fine-tuning. Figure 3 shows the main settings of this process.

We kept EPOCHS = 3 to avoid overfitting and to preserve the model’s original instruction-following ability. MAX_LENGTH = 1024 was a practical tradeoff: shorter inputs improved stability and speed, even if some long emails were truncated. Using BATCH_SIZE = 16 with GRAD_ACCUM = 1 let me take better advantage of the A100’s throughput, while keeping the update path straightforward. The LEARNING_RATE = 3e-5 was the most comforting choice, because a smaller step size helps protect the model’s general capability from being overwritten by the phishing task. With LORA_R = 16, LORA_ALPHA = 32, and LORA_DROPOUT = 0.05, we tried to keep

adaptation efficient but controlled. Overall, the process felt like balancing ambition with restraint: teaching the model to detect phishing without making it forget how to think.

```
MODEL_NAME = "Qwen/Qwen2.5-1.5B-Instruct"
OUTPUT_DIR = "./qwen2.5_1.5b_lora_a100"
EPOCHS = 3
MAX_LENGTH = 1024          # 依實際資料集長度調整
BATCH_SIZE = 16            # 提高 BATCH_SIZE 充分利用 A100 算力
GRAD_ACCUM = 1             # 配合 BATCH_SIZE 調整
LEARNING_RATE = 3e-5       # 調降學習率以保護 Instruct 模型的常規能力
LORA_R = 16
LORA_ALPHA = 32
LORA_DROPOUT = 0.05
SEED = 42
```

Figure 3: Main Setting of LoRA Fine-tuning

4 Results

4.1. Confusion Matrix

We use test.csv to test how our model performs. Table 3 shows the confusion matrix of the performance. We can see accuracy, precision, recall, and F1-score rise after fine-tuning, and also they have large increases except recall.

Recall score tells us how many phishing emails are detected by the model. Though it is comparatively high before fine-tuning, it increases about 0.1 after fine-tuning. Also, the FN decreases to 1, we can say that the model is good at identifying phishing emails.

On the other hand, the value of precision means that how many emails are real phishing in those that model thinks as phishing. The model obtained 474 FP before fine-tuning, that is, the model tends to classify emails to phishing. Since many legitimate emails would be seen as phishing, if we apply the model to the real world, this might cause users to be not aware of the alert since alert fatigue. However, it jumps from 0.5416 to 0.9984 (a gap about 0.45) after fine-tuning, and the FP is down to 1. Therefore, we can say that the performance in identifying phishing emails of fine-tuned model has significantly increased.

To conclude, fine-tuning can lower the false alarm rate, and increase the precision.

Table 3: Confusion Matrix of baseline and fine-tuning model

Model	ollama qwen2.5:1.5b	Qwen2.5-1.5B-Instruct after LoRA BF16 fine-tuning
Accuracy	0.5684	0.9960
Precision	0.5416	0.9984
Recall	0.8903	0.9936
F1-score	0.6735	0.9960
TP	560	625
TN	155	606
FP	474	1
FN	69	4

4.2. WebUI

Since multiple operating systems, such as Windows and macOS, are commonly used, deploying the system through a web browser is the most practical solution to ensure cross-platform compatibility. For this reason, we built the WebUI using google colab.

Figure 4 and 5 demonstrate how to use the WebUI. ⁵In simple terms, users could copy the subject and body and paste them to the system. After setting the threshold, users could input the contents to the fine-tuned LLM. Waiting for the process, it will output the risk score and level. Users could take actions by considering the risk score and level it provided.

In Figure 4, there are part (a) and (b). In part (a), this is the place where users paste the email contents. By deleting the former contents, users can paste new things. Next, set the threshold in part (b) that . After the steps before, click the orange button “啟動模型偵此與控制輸出” to start the LLM process.

⁵ Here is the video demonstrating how to use the system. It might be much more understanding.
https://drive.google.com/file/d/1Bn9_OrXNJO1dEVGxulVnasttunsR-Yqd/view?usp=sharing



Figure 4: Enter email contents and set the threshold

Continuously, in Figure 5, this is the output area. It shows the risk score in part (c) and the risk level in part (d). If the contents provided by users is considered not phishing, the risk score will be “0”, and the level will be “LOW(低風險)”. On the other hand, if it is phishing, the risk score will be “1” and the level will be “HIGH(高風險)”.

Through this system, users will be able to identify emails containing malicious incentives much more easily.

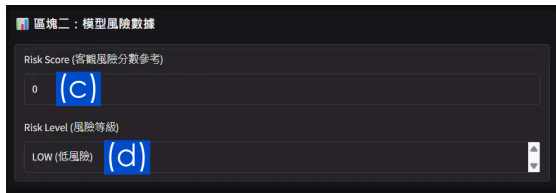


Figure 5: The result processed by LLM

5 Conclusion

In this project, we conducted a PoC experiment of detecting phishing emails through LLM, by building a LLM-based system which has the capability to defend against attacks from conventional and AI-generated phishing emails.

With the dataset that we integrated Enron, Nazario, and Adversarial BEC datasets, and using the LoRA fine-tuning method, the model successfully learnt the border of real communication and malicious contents. The precision of the fine-tuned model increases to 0.9984, and the false positive is down to 1. This result indicates that our parameter strategy can

effectively transform a general LLM into a model that possesses high precision and real-world value.

6 Limitations

Although the proposed system demonstrates excellent performance on the test set, the current architecture still has several practical limitations:

- **Text-only Limitation:** Since this paper is focused on the capability of detecting malicious incentive in text, the current system relies on textual content for feature extraction and classification. As a result, if there are any malicious QR codes included in phishing emails, the existing architecture is unable to parse and detect such threats.
- **Vulnerability to Prompt Injection Attacks:** Since the system is based on LLM, the model remains susceptible to adversarial prompt injection attacks.

References

- [1] Prashantha G R, Meghana G R, Bhoomika P M, Prajwal C T, Shivasali Rajendra Babu, 2023, “Detection Of Phishing Using Machine Learning,” INTERNATIONAL JOURNAL OF ENGINEERING RESEARCH & TECHNOLOGY (IJERT) Volume 11, Issue 05 (ICEI – 2023).
- [2] Shang-En Tsai, Chun-An Kuo, Wei-cheng Sun, 2025, Phishing Email Detection Based on Large Language Models,” Communications of the CCISA, Vol.31, No. 3.
- [3] T. Koide, H. Nakano and D.Chiba, 2024, “ChatPhishDetector: Detecting Phishing Sites Using Large Language Models,” in IEEE Access, vol. 12, pp. 154381-154400, doi: 10.1109/ACCESS.2024.3483905.
- [4] Hazell, J, 2024, “Spear phishing with large language models,” arXiv preprint arXiv:2305.06972v3
- [5] Opara, C., Modesti, P., & Golightly, L, 2025, “Evaluating spam filters and Stylometric Detection of AI-generated phishing emails,” Expert Systems with Applications, 276, Article 127044. <https://doi.org/10.1016/j.eswa.2025.127044>
- [6] Hoxhunt, “Phishing Trends Report (Updated for 2026),” Available: <https://hoxhunt.com/guide/phishing-trends-report>
- [7] Will Cukierski, 2015, “The Enron Email Dataset,” Kaggle, Available: <https://www.kaggle.com/datasets/wcukierski/enron-email-dataset/data>

- [8] J. Nazario, “Phishing email corpus,” Available:
<https://monkey.org/~jose/phishing/>
- [9] Yaw Osei Adjei, 2025, “Adversarial BEC Email Dataset,” Kaggle, Available:
<https://www.kaggle.com/datasets/yoadjei/adversarial-bec-email-dataset/data>
- [10] Jo-ting Chen, 2026, “cyber_LLM,” GitHub repository,
https://github.com/joting0518/cyber_LLM
- [11] 林佩璇, 2026, “介面demo影片”, Google Drive,
https://drive.google.com/file/d/1Bn9_OrXNJQ1dEVGxulVnasttunsR-Yqd/view?usp=sharing